

Guide to the K-Means Colab Exercises

Companion to `Exercises_KMeans.ipynb`

How to use this guide. Read the *key material* section before starting. Then, for each question, try on your own first. If you are stuck for more than a few minutes, open the hints in order: **Hint 1** points you in the right direction, **Hint 2** gives the method, **Hint 3** takes you to the last step. The hints never give the final numbers: the point is that you produce them. The *common mistakes* boxes list what goes wrong most often; read them even when you think you are done.

Key material

K-means in one paragraph

K-means looks for K centres $\mu_1, \dots, \mu_K$ and an assignment of each point to one centre that minimise the **inertia**

$$J = \sum_i \|x_i - \mu_{c(i)}\|^2.$$

Lloyd's algorithm alternates two steps (assign each point to its nearest centre; move each centre to the mean of its points) until nothing changes. Each step can only lower J , but the result is a *local* minimum that depends on the starting centres.

The KMeans object in scikit-learn

```
km = KMeans(n_clusters=3, n_init=10, random_state=0)
km.fit(X)                                # runs the algorithm n_init times, keeps the best
km.labels_                               # cluster of each training point
km.cluster_centers_                      # array (n_clusters, n_features)
km.inertia_                              # J of the best run
km.predict(X_new)                        # nearest centre for new points
km.fit_predict(X)                        # fit, then return labels_
```

`init="k-means++"` (the default) spreads the initial centres; `init="random"` picks random points.

Three ways to judge a clustering

- **Inertia** always decreases when K grows (more centres, closer to every point), so its minimum is useless for choosing K . Look for an *elbow*: the K after which extra clusters bring little.
- **Silhouette** of a point: $s = (b - a) / \max(a, b)$, where a is its mean distance to its own cluster and b its mean distance to the nearest other cluster. s is close to 1 for a well-placed point and negative for a misplaced one. `silhouette_score(X, labels)` averages it; higher is better, and it can be compared across K .
- **Adjusted Rand index** (ARI) compares a clustering with known labels: 1 is perfect agreement, about 0 is what random clusters get. It does not care about the numbering of clusters (cluster "2" does not need to be species "2"). Only usable when labels exist, i.e. for evaluation, never for fitting.

Mathematical insight · K -means as coordinate descent

The inertia depends on two groups of unknowns, the assignments c and the centres μ : $J(c, \mu) = \sum_i \|x_i - \mu_{c(i)}\|^2$. Lloyd's algorithm minimises J over c with μ fixed (nearest centre), then over μ with c fixed (the mean, because $\sum_{i \in S} \|x_i - \mu\|^2 = \sum_{i \in S} \|x_i - \bar{x}_S\|^2 + |S| \|\bar{x}_S - \mu\|^2$). Each half-step can only decrease J and there are finitely many partitions, so the algorithm stops. Finding the *global* minimum is NP-hard in general, which is why initialisation and restarts matter.

Mathematical insight · total, within and between sums of squares

With $\bar{x}$ the global mean and n_j the size of cluster j ,

$$\underbrace{\sum_i \|x_i - \bar{x}\|^2}_{\text{total (fixed)}} = \underbrace{\sum_j \sum_{i \in j} \|x_i - \mu_j\|^2}_{\text{within = inertia}} + \underbrace{\sum_j n_j \|\mu_j - \bar{x}\|^2}_{\text{between}}.$$

Minimising the inertia is the same as maximising the separation between centres. It also shows why the optimal inertia can only decrease with K : a $(K+1)$ -clustering can always copy the best K -clustering and split one cluster, which cannot increase the within term. With $K = n$ the inertia is 0.

Mathematical insight · the borders between clusters are straight

A point x is closer to μ_1 than to μ_2 when $\|x - \mu_1\|^2 < \|x - \mu_2\|^2$. Expanding, the $\|x\|^2$ terms cancel and the condition becomes

$$2(\mu_2 - \mu_1)^\top x < \|\mu_2\|^2 - \|\mu_1\|^2,$$

a linear inequality. The border is the perpendicular bisector of the two centres, and each cluster is an intersection of half-spaces: a convex region. Exercise 3 is a direct consequence.

Mathematical insight · K -means as a limit of Gaussian mixtures

Take a mixture of K Gaussians with equal weights and the same spherical covariance $\sigma^2 I$. The probability that point x belongs to component j is proportional to $\exp(-\|x - \mu_j\|^2 / 2\sigma^2)$. As $\sigma \rightarrow 0$ this becomes 1 for the nearest centre and 0 for the others, and the EM algorithm for the mixture becomes exactly Lloyd's algorithm. So K -means is the "hard, spherical, equal-size" special case of a Gaussian mixture; releasing these assumptions (full covariances, unequal weights, soft assignments) gives the `GaussianMixture` of Exercise 3.

Exercise 1 Clustering iris

1.1 Inertia and silhouette for $k = 2, \dots, 8$

Hint 1 · Nudge Standardise with `StandardScaler().fit_transform(X)`. Then one loop over k with two `append` calls.

Hint 2 · Method Inside the loop: `fit KMeans(n_clusters=k, n_init=10, random_state=0)` on `Xs`, `append km.inertia_`, and `append silhouette_score(Xs, km.labels_)`.

Hint 3 · Almost there `ks` is a range; wrap it in `list(ks)` for plotting. Use `plt.subplots(1, 2)` to draw the two curves side by side.

Mathematical insight · reading the silhouette

For point i , $a(i)$ is its mean distance to the other points of its cluster and $b(i)$ the smallest mean distance to the points of another cluster. $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$. Merging two clusters that touch (like versicolor and virginica) raises a a little for their points but raises b a lot for everyone else, because the “nearest other cluster” is now far away. That is why the silhouette often prefers fewer, well separated groups. Unlike inertia, the silhouette does not improve automatically with K , which is what makes it usable for model selection.

Question 1 (written)

Hint 1 · Nudge Find the k with the highest silhouette. Is it 3?

Hint 2 · Method Look at the petal plot: how many groups would *you* draw if you had no colours? Which two species touch?

Hint 3 · Almost there The silhouette rewards compact, well separated groups. It measures geometry, not biology. What else should inform the choice of k in practice?

Mathematical insight · what the adjusted Rand index corrects

The Rand index counts, over all $\binom{n}{2}$ pairs of points, the fraction of pairs on which two partitions agree (both put the pair together, or both separate it). Even two random partitions agree on many pairs, so the raw index is hard to read. The ARI subtracts the agreement expected by chance (for random partitions with the same cluster sizes) and rescales:

$$\text{ARI} = \frac{\text{index} - \mathbb{E}[\text{index}]}{\max \text{index} - \mathbb{E}[\text{index}]}$$

Hence $\text{ARI} \approx 0$ for random labels, 1 for identical partitions (up to renaming), and it can be slightly negative.

1.2 Scaled versus raw

Hint 2 · Method Two `fit_predict` calls with the same settings, one on `Xs`, one on `X`; then `adjusted_rand_score(species, labels)` for each.

Hint 3 · Almost there In the crosstab, rows are species and columns are clusters. A perfect clustering has exactly one non-zero entry per row and per column. Count the off-diagonal flowers and see which species are confused.

Question 2 (written)

Hint 1 · Nudge Compare the four standard deviations printed at the end of 1.2. Which feature dominates the raw distance?

Hint 2 · Method Is that dominant feature a good or a bad one for separating species? Look at the petal plot of 1.1.

Hint 3 · Almost there Standardising means “every feature matters equally”. When is that assumption justified, and when does it throw away useful information? Consider whether the features share a unit.

Common mistakes

- Using the species to choose k or to fit anything: labels are for the final evaluation only.
- Comparing cluster numbers with species numbers directly (“cluster 0 should be setosa”). Use ARI or a crosstab.
- Concluding “scaling is bad”. The lesson is narrower; say exactly what it is.

Exercise 2 Colour quantisation

An image as a dataset

A colour image of height H and width W is an array of shape $(H, W, 3)$. Reshaping it to $(H \cdot W, 3)$ gives one row per pixel and three features (R, G, B). K -means on these rows finds K representative colours (a *palette*). Replacing each pixel by its centre gives an image with only K colours: this is **vector quantisation**.

Storage: the original needs 24 bits per pixel. The quantised image needs $\lceil \log_2 K \rceil$ bits per pixel (an index into the palette) plus $24K$ bits for the palette itself.

Indexing tricks you will need

```
flat = img.reshape(-1, 3)           # (n_pixels, 3); -1 = "work it out"
rng = np.random.default_rng(seed)
idx = rng.choice(len(flat), size=1000, replace=False) # random row numbers
centres[labels] # labels: (n_pixels,) ints -> (n_pixels, 3) colours
arr.reshape(img.shape) # back to (H, W, 3)
```

Mathematical insight · rate and distortion

Vector quantisation trades *rate* (bits per pixel, $\approx \log_2 K$) against *distortion* (MSE). For large K and smooth data in dimension p , quantisation theory predicts that the distortion falls roughly like $K^{-2/p}$; here $p = 3$, so doubling the palette divides the MSE by about $2^{2/3} \approx 1.6$. Compare this prediction with your MSE column: real images are not smooth densities, but the order of magnitude is often right for the larger K .

2.2 quantize

Hint 1 · Nudge The four steps in the notebook translate to about five lines. Fit on a sample, predict on everything.

Hint 2 · Method Draw the sample rows with `rng.choice`, fit `KMeans(n_clusters=k, n_init=4, random_state=seed)` on them, then call `km.predict(flat)` to get one label per pixel.

Hint 3 · Almost there `km.cluster_centers_[labels]` gives the new colour of every pixel. Reshape it to `img.shape` and return it together with `km`.

2.3 Distortion versus size

Hint 2 · Method `MSE: ((img - q) ** 2).mean()`. Ratio: apply the formula in the notebook with `n = img.shape[0] * img.shape[1]` and `np.ceil(np.log2(k))`.

Hint 3 · Almost there `fig, axes = plt.subplots(2, 3)` and `zip(axes.ravel(), k_list)` let you fill the grid in one loop. `ax.imshow(q)` and `ax.axis("off")` for each image.

Question 3 (written)

Hint 1 · Nudge Look at both the images and the MSE curve. Where does the curve start to flatten?

Hint 2 · Method Relate MSE to inertia: inertia sums squared distances over pixels and the 3 colour channels; MSE averages the same squares. What is the factor between them?

Hint 3 · Almost there Justify your choice of K with a trade-off sentence: “going from __ to __ colours reduces MSE by only __ but costs __ more bits per pixel”.

Mathematical insight · what k -means++ guarantees

Arthur and Vassilvitskii (2007) proved that the k -means++ seeding alone (before any Lloyd iteration) gives an expected inertia at most $8(\ln K + 2)$ times the optimum. Random seeding has no such guarantee: two initial centres in the same dense region can leave a whole group of points without a centre, a local minimum Lloyd’s algorithm cannot escape. The $D(x)^2$ sampling makes that unlikely because points far from all current centres are favoured.

2.4 Initialisation

Hint 2 · Method Two lists, one loop over `s in range(10)`, two fits per iteration, differing only in `init`. Keep `n_init=1` so that you see each single run.

Hint 3 · Almost there `pd.DataFrame({...}).agg(["mean", "min", "max", "std"])` summarises both columns at once.

Question 4 (written)

Hint 1 · Nudge Compare both the *average* and the *spread* of the two columns.

Hint 2 · Method Why can two runs of the same algorithm on the same data end at different inertias? What does `n_init` do with several runs?

Exercise 3 When K -means fails

The geometry behind K -means

Each point goes to its nearest centre, so cluster j is the set of points closer to μ_j than to any other centre: a **Voronoi cell**. These cells are convex and their borders are straight lines (hyperplanes). Consequences:

- clusters that are stretched in one direction get cut across;
- clusters that are curved or wrapped around each other cannot be separated by straight borders.

A **Gaussian mixture** (`GaussianMixture`) models each cluster as a Gaussian with its own mean and covariance matrix, so it can learn *elliptical* shapes. Its components are still convex blobs.

3.1 Stretched clusters

Hint 2 · Method `KMeans(...).fit_predict(Xa)`. For the mixture: `GaussianMixture(3, covariance_type="full", random_state=0).fit(Xa).predict(Xa)`. ARI of each against `ya`.

Hint 3 · Almost there Two scatter plots side by side, coloured by the predicted labels; put the ARI in each title.

3.2 Moons

Hint 2 · Method Same pattern as 3.1 with `KMeans(2, ...)` on `Xm`, ARI against `ym`.

Question 5 (written)

Hint 1 · Nudge On your plots, draw (mentally) the border between the K -means clusters. Is it straight?

Hint 2 · Method For 3.1: what does K -means' distance ignore that the full covariance matrix captures? For 3.2: is there any straight line that separates the two moons?

Hint 3 · Almost there Look up DBSCAN and spectral clustering in the scikit-learn user guide (section "Clustering"): what do they use instead of distance to a centre?

Check yourself before moving on

- Why can you not choose K by minimising the inertia?
- What is the difference between what the silhouette measures and what the ARI measures?
- In colour quantisation, what exactly is being compressed and what is lost?
- Which cluster shapes can K -means recover, and why?